Loops

Mahir Labib Dihan
Lecturer, CSE, BUET

Table of Contents

The Philosophy of Repetition

Motivation

Scenario: Print "The sky is the limit!"

```
int main() {
    printf("The sky is the limit!");
}
```

Motivation

Scenario: Print "The sky is the limit!" *60 times.*

```
int main() {
    printf("The sky is the limit!");
    printf("The sky is the limit!");
    printf("The sky is the limit!");
    ... (Write this 60 times?)
    printf("The sky is the limit!");
    printf("The sky is the limit!");
}
```

Loops / Repetition Statements

Definition:

- *Repetition statements* allow us to execute a statement multiple times.
- Often they are referred to as *loops.*

Scenario: Print "The sky is the limit!" *60 times.*

Without Loops:

- Write the logic once.
- Copy-paste it 60 times.

With Loops:

- Write the logic once.
- Tell the computer to repeat it 60 times.

Rule of thumb: If you are copy-pasting code more than twice, you likely need a loop.

Types of Loops in C

C has three kinds of repetition statements:

- the *while* loop
- the *for* loop
- the *do-while* loop

Key Insight: All three loops are interchangeable in functionality, but choosing the right one improves code readability

The `while` Loop

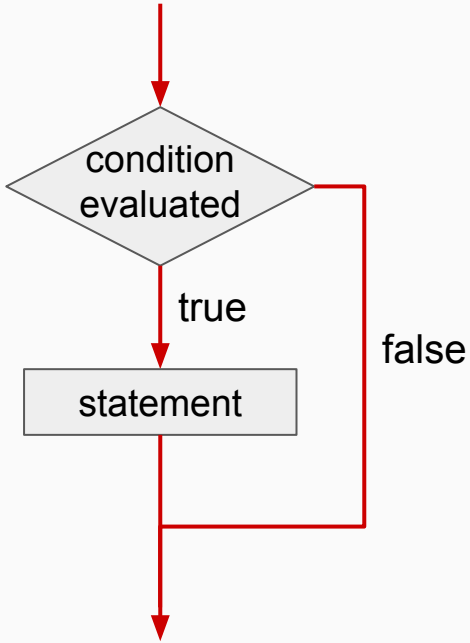
The `while` Loop

যতক্ষণ যদি

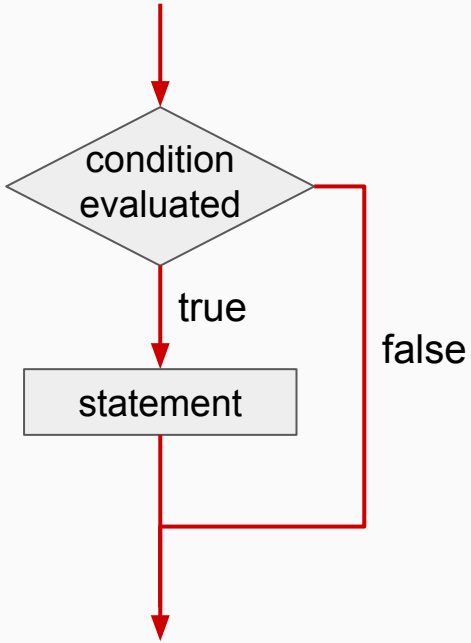
```
while if ( condition )
          statement;
```

- **`if`** condition is satisfied execute the statement(s)
- **`while`** condition is satisfied execute the statement(s)

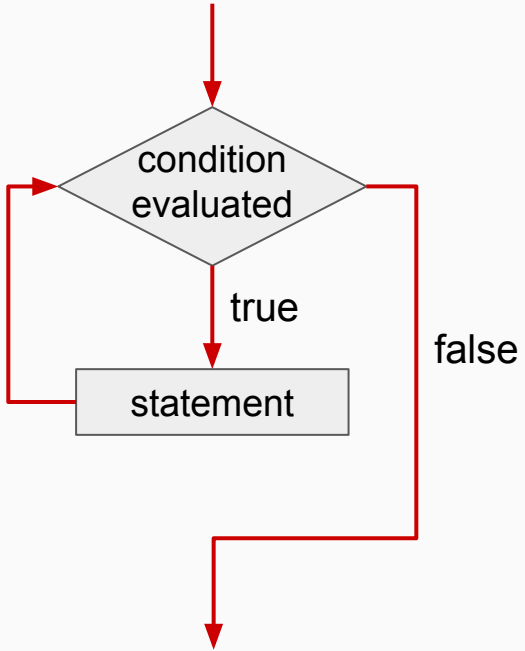
Flowchart: if Statement



Flowchart: while Loop



if Logic



The while Loop

The `while` Loop: Syntax

- A while loop/statement has the following syntax:

```
while ( condition )
    statement;
```

```
while ( condition ) {
    statement1;
    statement2;
    ...
}
```

- If the *condition* is true, the *statement* or *a block of statements* is executed
- Then the condition is evaluated again, and if it is still true, the statement/block is executed again
- The statement/block is executed repeatedly until the condition becomes false

Example: Positive Number

Task: Write a program that continuously asks for positive number as input.

```
int n;
printf("Please enter a positive number: ");
scanf("%d", &n);
while (n < 0) {
    printf ("Enter positive number, BE POSITIVE: ");
    scanf("%d", &n);
}
```

Output:

```
Please enter a positive number: -10
Enter positive number, BE POSITIVE: -5
Enter positive number, BE POSITIVE: -9
Enter positive number, BE POSITIVE: 10
```

Example: Repetitive Print

Task: Print “The sky is the limit!” 60 times.

```
int n = 60;
int count = 0;
while (count < n)
{
    printf("The sky is the limit");
    count++;
}
```

Example: Repetitive Print

Task: Print “The sky is the limit!” n times. n will be user input.

```
int n;
scanf("%d", &n);
int count = 0;   // initialization
while (count < n)
{
    printf("The sky is the limit");
    count++;     // update
}
```

- If the condition of a `while` loop is false initially, the statement is never executed.
- Therefore, the body of a while loop will execute **zero or more** times.

Example: Repetitive Print

Task: Print “The sky is the limit!” n times. n will be user input.

```
int n;
scanf("%d", &n);
int count = 1;    // initialization
while (count <= n)
{
    printf("The sky is the limit");
    count++;      // update
}
```

The while Loop: Summary

A while loop is functionally equivalent to the following structure:

```
initialization;

while (condition) {
    statement;
    update;
}
```

The for Loop

The for Loop: Syntax

- A for loop/statement has the following syntax:

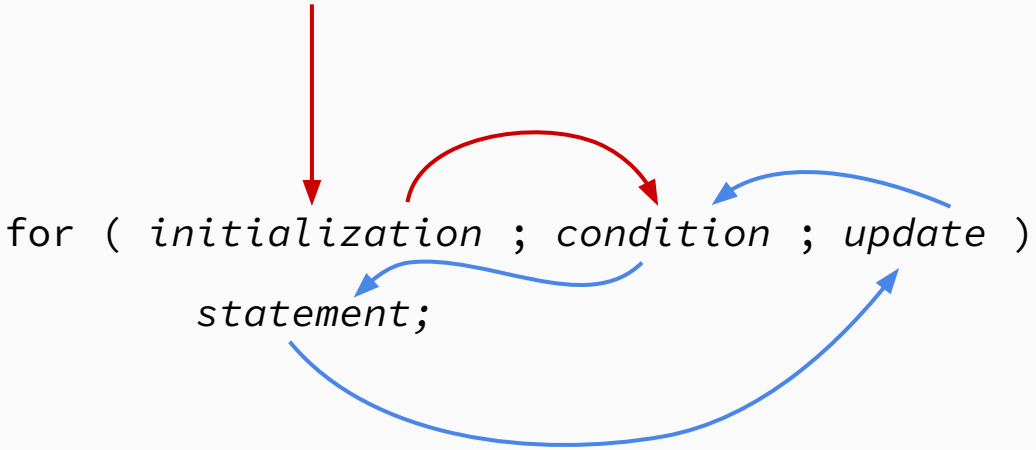
The *initialization* is executed once before the loop begins

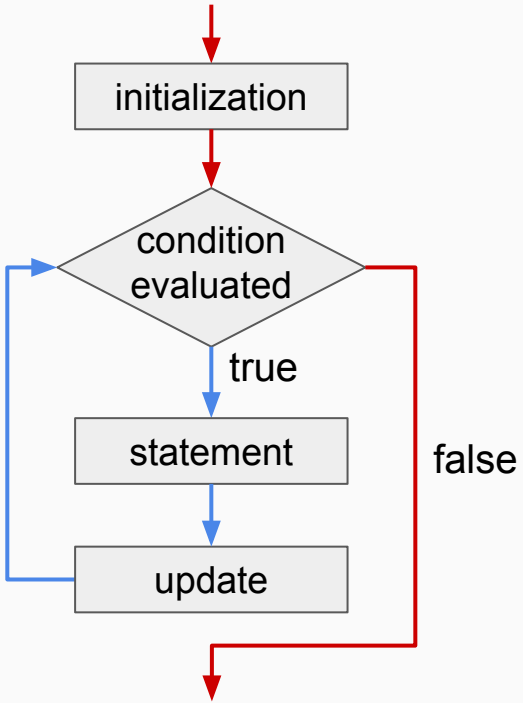
The *statement* is executed until the *condition* becomes false

```
for ( initialization ; condition ; update )
    statement;
```

The *update* portion is executed at the end of each iteration

Flowchart: for Loop





The for Loop: Example

An example of a for loop:

```
for (count=1; count <= n; count++)
    printf ("%d\n", count);
```

```
for (count=n; count >= 1; count--)
    printf ("%d\n", count);
```

- The initialization section can be used to declare a variable
- Like a `while` loop, the condition of a `for` loop is tested prior to executing the loop body
- Therefore, the body of a `for` loop will execute *zero or more* times

The for Loop: Example

The *update* section can perform any calculation

```
int num;
for (num=100; num > 0; num -= 5)
    printf ("%d\n", num);
```

- A for loop is well suited for executing statements a specific number of times that can be calculated or determined in advance

Example: Sum of Series

Write down a program to find the summation of the following series:

$$t_1 + t_2 + t_3 + t_4 + \ldots + t_n$$

$t_i = f(i)$

```
int main() {
     int i, n, t, s = 0;
     scanf("%d", &n);
     for(i = 1; i <= n; i++){
          ti = f(i)
          s = s + ti
     }
     printf("%d", s);
}
```

Example 1: Sum of Natural Numbers

Write down a program to find the summation of the following series:

$$1 + 2 + 3 + 4 + \ldots + n$$

$t_i = i$

```
int main() {
    int i, n, t, s = 0;
    scanf("%d", &n);
    for(i = 1; i <= n; i++){
        t = i
        s = s + t
    }
    printf("%d", s);
}
```

Example 1: Sum of Natural Numbers

Write down a program to find the summation of the following series:

$$1 + 2 + 3 + 4 + \ldots + n$$

$t_i = i$

```
int main() {
     int i, n, t, s = 0;
     scanf("%d", &n);
     for(i = 1; i <= n; i++){
          s = s + i
     }
     printf("%d", s);
}
```

Example 2: Sum of Square Numbers

Write down a program to find the summation of the following series:

$$1^2 + 2^2 + 3^2 + 4^2 + \ldots + n^2$$

i = 1 i = 2 i = n

$$t_i = i^2$$

```
int main() {
    int i, n, t, s = 0;
    scanf("%d", &n);
    for(i = 1; i <= n; i++){
        t = i*i
        s = s + t
    }
    printf("%d", s);
}
```

Example 3

Write down a program to find the summation of the following series:

$$1^2 - 2^2 + 3^2 - 4^2 + \ldots \text{ up to } n^2$$

$t_i = i^2$ when i is odd

$t_i = -i^2$ when i is even

```
int main() {
     int i, n, t, s = 0;
     scanf("%d", &n);
     for(i = 1; i <= n; i++){
          if(i%2 == 1) t = i*i;
          else  t = -i*i;
          s = s + t
     }
     printf("%d", s);
}
```

Example 4: Factorial Calculation

Print factorial of n:

$$n! = 1 \times 2 \times 3 \times 4 \times \ldots \times n^2$$

$t_i = i$

```
int main() {
     int i, n, t, p = 1;
     scanf("%d", &n);
     for(i = 1; i <= n; i++){
          t = i
          p = p * t
     }
     printf("%d", p);
}
```

Example 5: Power Calculation

Print x^n:

$$x^n = x \times x \times x \times x \times \ldots \times x$$

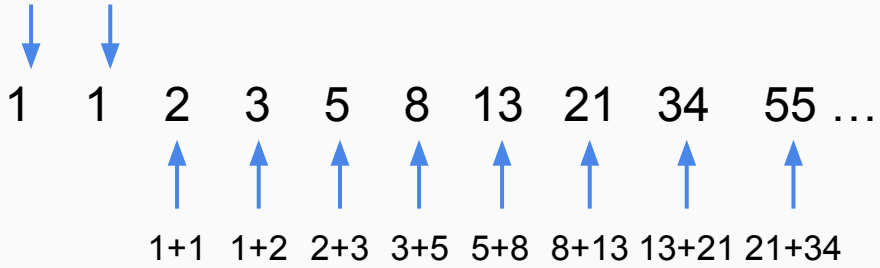
i = 1 i = 2 i = n

$t_i = x$

```
int main() {
    int i, n, t, p = 1;
    scanf("%d", &n);
    for(i = 1; i <= n; i++){
        t = x
        p = p * t
    }
    printf("%d", p);
}
```

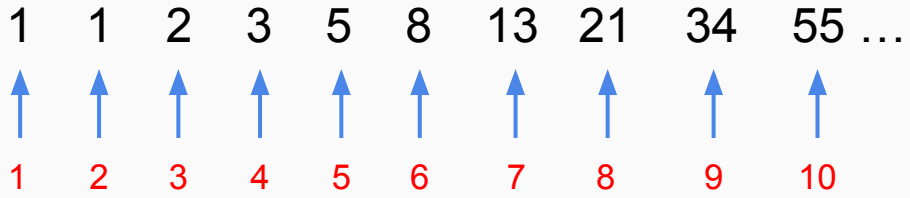
Example 6: Fibonacci Series

The **first** and **second** numbers in the Fibonacci sequence are 1



The **Fibonacci series** is a sequence of numbers where each term is the sum of the two previous terms

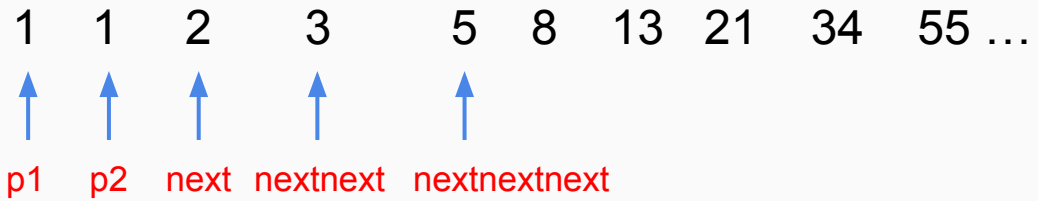
Example 6: Fibonacci Series



Write down a program that will print **n-th Fibonacci** number where n will be input to your program.

- n = 4; output → 3
- n = 7; output → 13

Example 6: Fibonacci Series




```
int main(){
     int p1, p2, next, n;
     scanf("%d", &n);
     p1 = 1;
     p2 = 1;
     next = p1 + p2;
     nextnext = p2 + next;
     nextnextnext = next + nextnext;
     ...
     ...
}
```

Example 6: Fibonacci Series


```
int main(){
      int p1, p2, next, n;
      scanf("%d", &n);
      p1 = 1;
      p2 = 1;
      for(i = ; i <= ; i++){
            next = p1 + p2;
            p1 = p2;
            p2 = next;
      }
      printf("%d", next);
}
```

Example 6: Fibonacci Series


```
int main(){
      int p1, p2, next, n;
      scanf("%d", &n);
      p1 = 1;
      p2 = 1;
      for(i = 3; i <= n; i++){
            next = p1 + p2;
            p1 = p2;
            p2 = next;
      }
      if(n <= 2) printf("%d", p1);
      else printf("%d", next);
}
```

The for Loop: Optional Parts

- Each expression in the header of a for loop is optional

```
for ( ; ; ) {
    // statements
}
```

- If the *initialization* is left out, no initialization is performed
- If the *condition* is left out, it is always considered to be true
- If the *update* is left out, no update operation is performed

Infinite Loops

- The body of a `while` loop eventually must make the condition false
- If not, it is called an *infinite loop*, which will execute until the user interrupts the program
- This is a common logical error
- You should always double check the logic of a program to ensure that your loops will terminate normally

Infinite Loops

An example of an infinite loop

```
int count = 1;
while (1 == 1) {
     printf("%d\n", count);
     count = count - 1;
}
```

```
int count = 1;
for ( ; ; ) {
     printf("%d\n", count);
     count = count - 1;
}
```

This loop will continue executing until interrupted (CTRL-C)

The do-while Loop

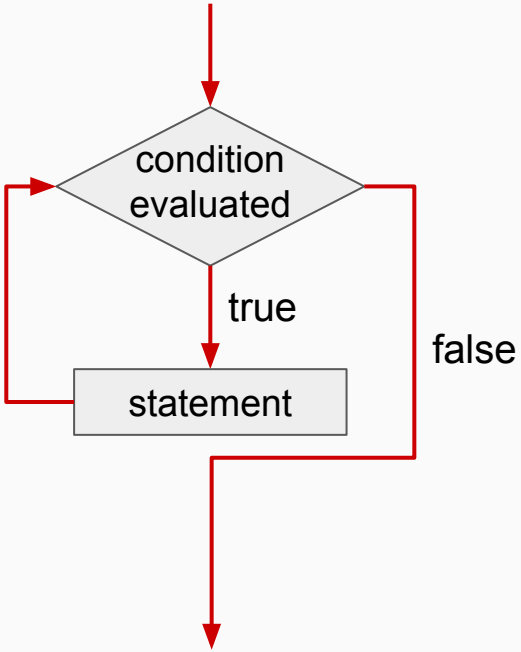
The do-while Loop: Syntax

- A do-while loop/statement has the following syntax:

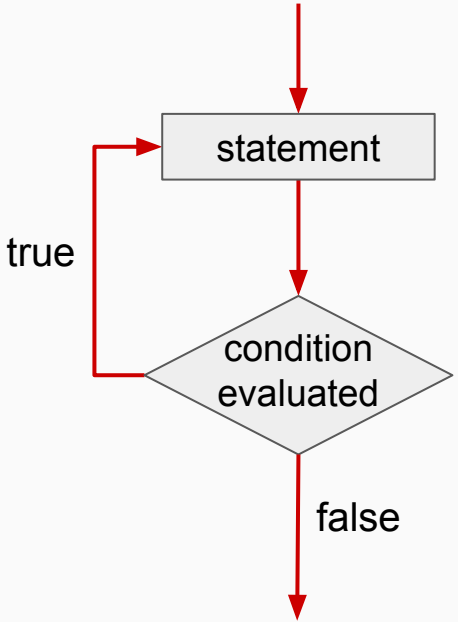
```
do {
    statement;
}
while ( condition );
```

- The statement is executed once initially, and then the condition is evaluated
- The statement is executed repeatedly until the condition becomes false

Flowchart: do-while Loop



The while Loop



The do-while Loop

The do-while Loop: Example

An example of a do-while loop:

```
int count = 1;
do{
    printf("%d\n", count);
    count++;
} while (count <= 5);
```

- The body of a `do-while` loop is executed **at least once**

The do-while Loop: Example

`while` loop:

```
int n;
printf("Please enter a positive number: ");
scanf("%d", &n);
while (n < 0) {
     printf ("Please enter a positive number: ");
     scanf("%d", &n);
}
```

`do-while` loop:

```
int n;
do {
     printf("Enter a positive number: ");
     scanf("%d",&n);
} while (n < 0);
```

Think First: while vs do-while

Question: What is the output of each?

Code A:

```
int x = 0;
while (x > 0) {
     printf("%d ", x);
     x--;
}
printf("End");
```

Answer A: `End`

(body executes once, then condition checked)

Code B:

```
int x = 0;
do {
     printf ("%d ", x);
     x--;
} while (x > 0);
printf("End");
```

Answer B: `0 End`

(body never executes, condition false initially)

Nested Loops

Nested Loops

- Similar to nested `if` statements, loops can be nested as well
- That is, the body of a loop can contain another loop
- For each iteration of the outer loop, the inner loop iterates completely

Nested Loops: Example 1

What will be the output?

```
for(i=1; i <= 3; i++) {
    for(j=1; j <= 2; j++) {
        printf("Sky is the limit\n");
    }
    printf("The world is becoming smaller\n");
}
```

Outer Loop

Inner Loop

Nested Loops: Example 2

What will be the output?

```
for(i=1; i <= 3; i++) {
    for(j=1; j <= 2; j++) {
        printf("Sky is the limit\n");
    }
    printf("The world is becoming smaller\n");
}
```

Output:

```
Sky is the limit
Sky is the limit
The world is becoming smaller
Sky is the limit
Sky is the limit
The world is becoming smaller
Sky is the limit
Sky is the limit
The world is becoming smaller
```

Nested Loops: Example 2

What will be the output?

```
for(i=1; i <= 3; i++){
     for(j=1; j <= 2; j++){
          printf("%d %d\n", i, j);
     }
}
```

Output:

```
1 1
1 2
2 1
2 2
3 1
3 2
```

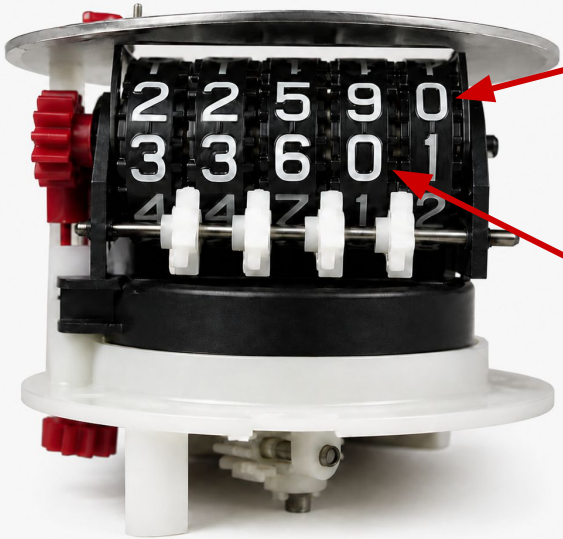
Nested Loops: Example 3

How many times will the string "Here" be printed?

```
count1 = 1;
while (count1 <= 10)
{
    count2 = 1;
    while (count2 <= 20)
    {
        printf("Here\n");
        count2++;
    }
    count1++;
}
```

10 * 20 = 200

Analogy of Nested Loops



Inner Loop

Outer Loop

Example: Printing a Rectangle

```
for (int row = 1; row <= 3; row ++) {
     for (int col = 1; col <= 5; col++) {
          printf ("* ");
     }
     printf("\n"); // Newline after each row
}
```

Output:

```
* * * * *
* * * * *
* * * * *
```

Total stars: $3 \times 5 = 15$

Example: Multiplication Table

```
for (int i = 1; i <= 5; i++) {
     for (int j = 1; j <= 5; j++) {
          printf("%4d", i * j); // %4d for alignment
     }
     printf("\n");
}
```

Output:

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```

Dependent Inner Loops: Right Triangle

What if the inner loop depends on the outer loop variable?

```
for (int i = 1; i <= 5; i++) {
     for (int j = 1; j <= i; j++) { // j goes up to i!
          printf("* ");
     }
     printf("\n");
}
```

Output:

```
*
* *
* * *
* * * *
* * * * *
```

Pattern: Number Triangle

```
for (int i = 1; i <= 5; i++) {
     for (int j = 1; j <= i ; j++) {
          printf("%d", j);
     }
     printf("\n");
}
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Think First: Inverted Triangle

Question: What code produces this output?

```
* * * * *
* * * *
* * *
* *
*
```

Answer:

```
for (int i = 5; i >= 1; i--) {
     for (int j = 1; j <= i ; j++) {
          printf("* ");
     }
     printf("\n");
}
```

Pyramid Pattern

Question: Print a centered pyramid with n rows.

```
    *
   ***
  *****
 *******
*********
```

Pyramid Solution

```
int n = 5;
for (int i = 1; i <= n ; i++) {
    // Print spaces
    for (int s = 1; s <= ??; s++) {
        printf(" ");
    }
    // Print stars
    for (int j = 1; j <= ??; j++) {
        printf("*");
    }
    printf("\n");
}
```

Pyramid Analysis

	1	2	3	4	5	6	7	8	9
1					*				
2				*	*	*			
3			*	*	*	*	*		
4		*	*	*	*	*	*	*	
5	*	*	*	*	*	*	*	*	*

Row	Spaces	Stars
1	4	1
2	3	3
3	2	5
4	1	7
5	0	9

Can we express the number of spaces and stars as a function of row number and n?

Pyramid Analysis

	1	2	3	4	5	6	7	8	9
1					*				
2				*	*	*			
3			*	*	*	*	*		
4		*	*	*	*	*	*	*	
5	*	*	*	*	*	*	*	*	*

Row	Spaces	Stars
1	5 - 1	2*1 - 1
2	5 - 2	2*2 - 1
3	5 - 3	2*3 - 1
4	5 - 4	2*4 - 1
5	5 - 5	2*5 - 1
i	n - i	2*i - 1

Pyramid Solution

```
int n = 5;
for (int i = 1; i <= n ; i++) {
    // Print spaces
    for (int s = 1; s <= n - i; s++) {
        printf(" ");
    }
    // Print stars
    for (int j = 1; j <= 2*i - 1; j++) {
        printf("*");
    }
    printf("\n");
}
```

Question	My Answer

CSE 273
 	
1. Write a program to print the pattern.
1
4 6
9 11 13
17 19 21 23
	`#include <stdio.h>`
	`int main()`
	`{`
	`  printf("1\n");`
	`  printf("4 6\n");`
	`  printf("9 11 13\n");`
	`  printf("17 19 21 23");`
	`  return 0;`
	`}`

Moral: This is why we learn loops!

(Hardcoding works... until the pattern changes to 100 rows)

Jump Statements: break & continue

The break and continue Statement

- Sometimes we need:
  - to skip some statements inside the loop (continue)
  - or terminate the loop immediately without checking the test condition (break)
- In such cases, break and continue statements are used

The break Statement

The break statement terminates the loop immediately when it is encountered

```
while (condition) {
      // statements
      if (condition to break) {
            break;
      }
      // statements
}
```

```
do {
      // statements
      if (condition to break) {
            break;
      }
      // statements
} while (condition);
```

```
for (init; condition; update) {
      // statements
      if (condition to break) {
            break;
      }
      // statements
}
```

The break Statement: Example

```c
// Program to calculate the sum of maximum of 10 numbers
// If negative number is entered, loop terminates, sum is displayed
int main( ) {
     int i;
     double number, sum = 0.0;
     for(i=1; i <= 10; i++) {
          printf("Enter n%d: ", i);
          scanf("%lf", &number);
          // If user enters negative number, loop is terminated
          if(number < 0.0) {
               break;
          }
          sum += number;
     }
     printf("Sum = %.2lf", sum);
}
```

The continue Statement

The break statement terminates the loop immediately when it is encountered

```
while (condition) {
      // statements
      if (condition) {
            continue;
      }
      // statements
}
```

```
do {
      // statements
      if (condition to break) {
            continue;
      }
      // statements
} while (condition);
```

```
for (init; condition; update) {
      // statements
      if (condition to break) {
            continue;
      }
      // statements
}
```

The continue Statement: Example

```c
// Program to calculate the sum of maximum of 10 +ve numbers
// If negative number is entered, it is ignored
int main() {
     int i;
     double number, sum = 0.0;
     for(i=1; i <= 10; i++) {
          printf(" Enter n%d: ", i);
          scanf("%lf", &number);
          // if user enters negative number, skip it
          if(number < 0.0) {
               continue;
          }
          sum += number;
     }
     printf("Sum = %.2lf", sum);
}
```

The break and continue Statement

- Sometimes we need:
  - to skip some statements inside the loop (continue)
  - or terminate the loop immediately without checking the test condition (break)
- In such cases, break and continue statements are used
- **Important:** In case of nested loops, break and continue only affect the **innermost** loop they're in!

Example: break - Finding First Divisor

```
int n = 100;
printf("First divisor of %d (other than 1): ", n);

for (int i = 2; i <= n ; i++) {
     if (n % i == 0) {
          printf("%d\n", i);
          break; // Found it , stop searching!
     }
}
```

Output: First divisor of 100 (other than 1): 2

Without break: Would print ALL divisors (2, 4, 5, 10, 20, 25, 50, 100)

Example: continue - Skip Specific Values

Print 1 to 10, but skip multiples of 3:

```
for (int i = 1; i <= 10; i++) {
     if (i % 3 == 0) {
          continue; // Skip rest of body, go to i++
     }
     printf("%d ", i);
}
```

Output: `1 2 4 5 7 8 10`

(Skipped: `3, 6, 9`)

Quiz

What is the output?

```
int i = 0;
while (i < 5) {
      if (i == 2) {
            continue;
      }
      printf("%d ", i);
      i++;
}
```

(a) 0 1 3 4 5

(b) 0 1 3 4

(c) 1 3 4 5

(d) Infinite Loop ✓

Nested Loops: break Only Exits Innermost

```
for (int i = 1; i <= 3; i++) {
     printf("Outer %d: ", i);
     for (int j = 1; j <= 5; j++) {
          if (j == 3) break; // Only breaks inner loop
          printf("%d ", j);
     }
     printf("\n");
}
```

Output:

```
Outer 1: 1 2
Outer 2: 1 2
Outer 3: 1 2
```

The outer loop continues! Each inner loop stops at `j=3`.

Breaking Out of Multiple Loops

Problem: How to exit all loops at once?

Solution 1: Use a flag variable

```
int found = 0;
for (int i = 0; i < 10 && !found; i++) {
     for (int j = 0; j < 10 && !found; j++) {
          if (arr[i][j] == target) {
               printf ("Found at (%d, %d)\n", i, j);
               found = 1; // This will stop both loops
          }
     }
}
```

Solution 2: Use `goto` (not recommended in general)

Practical Examples

Example: Sum of Digits

Problem: Find the sum of digits of a number (e.g., 1234 → 1+2+3+4 = 10)

```
int num = 1234, sum = 0;

while (num > 0) {
     int digit = num % 10; // Extract last digit
     sum = sum + digit ; // Add to sum
     num = num / 10; // Remove last digit
}

printf("Sum of digits: %d\n", sum);
```

Trace:

- num=1234: digit=4, sum=4, num=123
- num=123: digit=3, sum=7, num=12
- num=12: digit=2, sum=9, num=1
- num=1: digit=1, sum=10, num=0
- num=0: Condition fails, output 10

Example: Count Digits in a Number

Problem: How many digits are in a given number?

```
int num, count = 0;
printf("Enter a number: ");
scanf("%d", &num);

// Handle special case of 0
if (num == 0) {
     count = 1;
} else {
     int temp = num;
     if (temp < 0) temp = -temp ; // Handle negative

     while (temp > 0) {
          count++;
          temp = temp / 10;
     }
}
printf("Number of digits: %d\n", count);
```

Example: Reverse a Number

Problem: Reverse the digits (e.g., 1234 → 4321)

```
int num = 1234, reversed = 0;

while (num > 0) {
     int digit = num % 10;
     reversed = reversed * 10 + digit;
     num = num / 10;
}

printf("Reversed: %d\n", reversed);
```

Think: What happens with num = 1200?

Answer: Output is 21 (leading zeros are dropped in integers!)

Example: Prime Checker

Problem: Check if a given number is prime

Definition
A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself.

Examples:

- Prime: $2, 3, 5, 7, 11, 13, 17, 19, 23, \ldots$
- Not Prime: 1 (by definition), 4 (2×2), 6 (2×3), 9 (3×3), …

Key Observations:

- 2 is the only even prime number
- To check if n is prime, we need to verify no number from 2 to n − 1 divides n

Example: Prime Checker (Naive)

Approach: Check if any number from 2 to n-1 divides n.

```
int isPrime = 1, i;
for (i = 2; i <= n - 1; i++) { // Check all numbers from 2 to n -1
     if (n % i == 0) {
          isPrime = 0;
          break; // Found a factor , no need to check more
     }
}
if (isPrime && n > 1)
     printf("%d is Prime\n", n);
else // 0 and 1 are not prime
     printf("%d is not Prime\n", n);
```

- For `n = 100`, we check: 2, 3, 4, 5, ..., 99 (98 checks!)
- For `n = 1000000`, we do 999,998 checks!
- Can we do better?

Optimization 1 – Check Only Up to $\sqrt{n}$

Key Insight: If $n = a \times b$ and $a \leq b$, then $a \leq \sqrt{n}$.

Example:

$$
\begin{aligned}
36 &= 2 \times 18 \\
&= 3 \times 12 \\
&= 4 \times 9 \\
&= 6 \times 6
\end{aligned}
$$

Optimization 1 – Check Only Up to $\sqrt{n}$

Key Insight: If $n = a \times b$ and $a \leq b$, then $a \leq \sqrt{n}$.

```
int isPrime = 1, i;
for (i = 2; i * i <= n; i++) { // Only check up to sqrt(n)
     if (n % i == 0) {
          isPrime = 0;
          break; // Found a factor , no need to check more
     }
}
```

Improvement:

- For `n = 100`, we check: 2, 3, ..., 10 (only 9 checks!)
- For `n = 1000000`, we check up to 1000 (only 999 checks!)

Optimization 2 – Skip Even Numbers

Key Insight : 2 is the only even prime. All other even numbers are not prime.

```
int isPrime = 1, i;
if (n <= 1) isPrime = 0;
else if (n > 2 && n % 2 == 0) isPrime = 0;
else
     for (i = 2; i * i <= n ; i++) { // Only check up to sqrt(n)
          if (n % i == 0) {
               isPrime = 0;
               break; // Found a factor , no need to check more
          }
     }
```

Optimization 2 – Skip Even Numbers

Key Insight: An odd number cannot be divided by an even number.

- even × even = even
- even × odd = even
- odd × odd = odd

Further Improvement: We now check only half the numbers: 3, 5, 7, 9, ...

```
int isPrime = 1, i;
if (n <= 1) isPrime = 0;
else if (n > 2 && n % 2 == 0) isPrime = 0;
else
     for (i = 3; i * i <= n ; i += 2) { // Only check up to sqrt(n)
          if (n % i == 0) {
               isPrime = 0;
               break; // Found a factor , no need to check more
          }
     }
```

Example: Finding GCD (Euclidean Algorithm)

Problem: Find GCD of two numbers using Euclid's method.

Algorithm: $\gcd(a, b) = \gcd(b, a \bmod b)$ until $b = 0$

```
int a = 48, b = 18;

while (b != 0) {
     int temp = b;
     b = a % b;
     a = temp;
}

printf("GCD is %d\n", a);
```

Trace:

- a=48, b=18: temp=18, b=48%18=12, a=18
- a=18, b=12: temp=12, b=18%12=6, a=12
- a=12, b=6: temp=6, b=12%6=0, a=6
- b=0: Exit. GCD = 6

Problem: Prime Numbers in Range

Print all prime numbers between 2 and 50:

```
for (int n = 2; n <= 50; n++) {
      int isPrime = 1;

      for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                  isPrime = 0;
                  break;
            }
      }

      if (isPrime) {
            printf("%d ", n);
      }
}
```

Output: `2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 50`

Problem: Diamond Pattern

Goal: Print a diamond with `n=5`

```
    *
   ***
  *****
 *******
*********
 *******
  *****
   ***
    *
```

Hint: Combine pyramid (top half) with inverted pyramid (bottom half)

Summary & Best Practices

Loop Selection Guide

Situation	Recommended Loop
Known number of iterations	`for`
Unknown iterations, might be 0	`while`
Must execute at least once	`do-while`
Menu-driven programs	`do-while`
Searching/finding first match	`for`/`while` with `break`

Common Mistakes to Avoid

1. **Infinite loops:** Forgetting update step
2. **Off-by-one errors:** < vs <=
3. **`continue` in `while`:** May skip increment
4. **Breaking wrong loop:** break only exits innermost
5. **Uninitialized variables:** Always initialize loop counters

Best Practices

1. **Initialize all variables** before using them
2. **Use meaningful names:** `row`, `col` instead of `i`, `j` when appropriate
3. **Use braces always**, even for single statements
4. **Limit nesting depth:** More than 3 levels is hard to read
5. **Comment complex loops:** Explain what pattern is being generated
6. **Test boundary conditions:** 0 iterations, 1 iteration, max value

List of all Keywords in C Language

Keywords in C Programming			
auto	break	case	char
const	**continue**	default	**do**
double	else	enum	extern
float	**for**	**goto**	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	**while**

Summary

1. **`while`:** Entry-controlled. Use when iterations unknown.
2. **`for`:** Best for counting. All parts (init, cond, update) in header.
3. **`do-while`:** Exit-controlled. Guarantees at least one execution.
4. **Nested loops:** Inner completes fully for each outer iteration. Think rows/columns.
5. **`break`:** Exit loop immediately.
6. **`continue`:** Skip to next iteration.

Acknowledgement

- Dr. Ashikur Rahman, Professor, CSE, BUET
- Mohammad Sadat Hossain, Lecturer, CSE, BUET